

2026-05-08-phase5-end-user-materials-design

Phase 5 — End-User Materials Uplift Design Spec

Date: 2026-05-08 **Author:** Claude Opus 4.7 **Status:** DRAFT (awaiting user review) **Successor:** to be handed to `superpowers:writing-plans` for executable plan **Phase:** 5 of CLI-Agent Fusion programme

1. Goals, non-goals, success criteria

1.1 What we’re building

A staged uplift of all end-user facing materials for HelixCode — documentation, installers/ packaging, and website — closing the gaps identified in the existing `USER_MANUAL_EXPANSION_PLAN.md` and adding delivery mechanisms that let end users install and evaluate the product with minimal friction.

1.2 Existing materials assessment

Category	Contents	State
Reference docs	8 <code>COMPLETE_*.md</code> files (API, CLI, Config, Deployment, Examples, Performance, Security, Troubleshooting)	Comprehensive but need content audit for stale/outdated sections
User manual	<code>README.md</code> (3027 lines), <code>SUMMARY.md</code> (632 lines), <code>INDEX.md</code> (13K)	Mature v2.0 manual; expansion plan already defines gaps
Tutorials	8 tutorials (2,355 lines total)	Cover major features; may need updating
Examples	4 config YAML files (basic, provider, enterprise, multi-worker)	Good baseline
Website	Full HTML/CSS/JS site (80K <code>index.html</code>), courses, mobile sub-sites, nginx config, Containerfile, test scripts, <code>install-podman.sh</code>	Feature-rich but may have stale links/stats
Installers	Only <code>install-podman.sh</code> (macOS Podman)	Major gap — no Linux packages, Homebrew, Windows installer
Materials	<code>Encryption_Algs.md</code> , <code>LLMs_Optimization.md</code>	

Category	Contents	State
		Niche; may need integration into main docs
Package READMEs	35+ internal/*/README.md files	Developer-facing; not user-facing but usable as source truth

1.3 Gap analysis (from expansion plan + audit)

Documentation gaps: - Provider-specific deep-dives (Anthropic extended thinking, Gemini multimodal, AWS/Azure IAM setup) - Memory system guide (vector DB comparison, persistence config) - Tool sandboxing configuration - Web tools enablement guide - Developer guides (custom provider tutorial, new tool via MCP SDK) - Troubleshooting (SSH workers, LLM connection debugging, rate limit handling) - Cross-reference integrity (SUMMARY/INDEX may be out of sync with manual content)

Installer gaps: - No .deb or .rpm packages for Linux - No Homebrew formula for macOS - No Windows installer - Docker deployment docs exist but no streamlined end-user docker run one-liner documented - No install.sh that detects platform and runs the right installer

Website gaps: - Feature listing may be stale (new features since Nov 2025) - Links to GitHub may be broken (repo structure changed) - Provider count/stats may need updating - Manual section on website may not reflect all user manual content - Mobile/courses subsites may need content sync

1.4 Goals (priority order)

- **G1 — Complete documentation.** Close every gap in the expansion plan. No “TODO” or “Coming soon” sections in user-facing docs.
- **G2 — Installable on all major platforms.** Linux (deb/rpm), macOS (Homebrew), Windows (installer), plus streamlined Docker.
- **G3 — Website matches reality.** Stats, links, feature descriptions reflect the actual HelixCode codebase, not aspirational goals.
- **G4 — Cross-reference integrity.** SUMMARY, INDEX, tutorials, and website all agree on feature names, config keys, and CLI flags.

1.5 Non-goals (explicit out-of-scope)

- **N1.** Rewriting the user manual from scratch. The existing 3K-line manual is solid; we extend it.
- **N2.** Redesigning the website layout. Content updates only, not restructuring.
- **N3.** Video course production. The existing courses subsite has materials; we don’t produce new videos.
- **N4.** Writing package-level Go doc comments for every internal package. That’s a developer experience concern, not end-user facing.
- **N5.** Translating docs to other languages.

1.6 Success criteria

- **S1.** USER_MANUAL_EXPANSION_PLAN.md every checkbox ticked.

- **S2.** Installer exists for each of: Linux (deb), Linux (rpm), macOS (Homebrew), Windows (exe/msi).
 - **S3.** `install.sh` (or equivalent) detects platform and invokes correct installer.
 - **S4.** Website feature list, provider count, and stats match codebase reality (verified by `grep` of actual provider implementations).
 - **S5.** No 404 links from website to GitHub or docs.
 - **S6.** `SUMMARY.md` and `INDEX.md` consistent with `README.md` section headers.
-

2. Work packages

WP-DOC: Documentation uplift

D1 — Provider deep-dives - Anthropic/Claude: extended thinking config, prompt caching setup, 200K context usage patterns - Google Gemini: multimodal (images/video) usage, 2M token context, flash models - Enterprise: AWS Bedrock IAM auth, Azure Entra ID setup, region selection - Free providers: XAI Grok free tier, OpenRouter free models, Qwen quota limits - Add sections to `docs/user_manual/README.md` Part III

D2 — Advanced feature guides - Memory system: vector DB comparison (Zep, Weaviate, ChromaDB, Qdrant), persistence config, choosing the right backend - Tool sandboxing: safe shell execution config, sandbox restrictions, allowed commands - Web tools: enabling web search, configuring search providers, HTML fetch/parse pipeline - Add sections to `docs/user_manual/README.md` Part IV / Part V

D3 — Developer guides - “Creating a Custom Provider”: step-by-step tutorial with example provider implementation - “Adding a New Tool”: using the MCP SDK tool registry, dispatch, and testing - New tutorials or appendix entries in `docs/user_manual/tutorials/`

D4 — Troubleshooting - SSH worker common errors (key auth, host key verification, connection timeout) - LLM connection debugging (timeout, auth failure, rate limit, model not found) - Rate limit handling (per-provider backoff, queue management, fallback chains) - Add to `docs/user_manual/README.md` Part X

D5 — Cross-reference sync - Walk `SUMMARY.md` topic list against `README.md` actual sections; fix mismatches - Walk `INDEX.md` against both; fix mismatches - Verify 8 `COMPLETE_*.md` files reference the same config keys as actual `config/config.yaml` - Update Tutorial 1-8 if any feature names/CLI flags changed

WP-INST: Installers & packaging

I1 — Linux packages - `.deb` package (Debian/Ubuntu): control file, systemd service, man page, postinst/prerm scripts - `.rpm` package (Fedora/RHEL): spec file, systemd service, man page - Both: install binary to `/usr/local/bin/helixcode`, config to `/etc/helixcode/`, data to `/var/lib/helixcode/` - Docker-based build for reproducibility (no host toolchain needed)

I2 — macOS Homebrew formula - Formula in Homebrew/`helixcode` tap or core PR - Binary bottle distribution - Post-install message with quick-start instructions - Support both Intel and Apple Silicon

I3 — Windows installer - NSIS or WiX MSI installer - Add to `PATH`, create start menu entry, install config template - PowerShell detection script for minimum version check

I4 — Platform-agnostic install script - `install.sh` (or `install.ps1` for Windows): detect OS/arch, download appropriate package, verify checksum, install - Supports `--version`, `--dry-run`, `--prefix` flags - Modeled after `rustup/brew` one-liner pattern: `curl -fsSL https://helixcode.dev/install.sh | sh`

I5 — Docker streamlined - Update existing Containerfile to produce minimal production image - Document one-liner: `docker run -d -p 8080:8080 -v helixcode-data:/var/lib/helixcode helixcode/helixcode:latest` - Docker Compose for production (postgres + redis + helixcode)

WP-WEB: Website uplift

W1 — Content accuracy audit - Compare every feature listed in `index.html` against actual Go code (`internal/` package listing) - Update provider count from old to actual (verify via `grep` of provider implementations) - Update stats (lines of code, supported languages, etc.) via actual measurements - Fix any GitHub links that changed due to repo restructuring

W2 — Manual sync - Ensure `docs/manual/index.html` reflects current `docs/user_manual/README.md` content - Update courses subsite (`docs/courses/`) if curriculum has changed - Update mobile subsite (`docs/mobile/`) if mobile client info has changed

W3 — Test suite run - Run `test-local.sh`, `test-website.sh`, `test-performance.sh` - Fix any failures - Verify containerized deployment via Containerfile

3. Dependencies and ordering

WP-DOC (docs first — content is foundation)

- └─ D1 (provider deep-dives)
- └─ D2 (advanced features)
- └─ D3 (developer guides)
- └─ D4 (troubleshooting)
- └─ D5 (cross-reference sync — runs last, touches everything)

WP-INST (installers — depends on knowing what to ship)

- └─ I1 (Linux packages)
- └─ I2 (macOS Homebrew)
- └─ I3 (Windows installer)
- └─ I4 (platform-agnostic `install.sh`)
- └─ I5 (Docker streamline — can run in parallel with I1-I4)

WP-WEB (website — depends on docs content being finalized)

- └─ W1 (content audit — can start early)
- └─ W2 (manual sync)
- └─ W3 (test suite — final verification)

Recommended execution order: 1. D1-D4 in parallel (independent content areas) 2. D5 (cross-ref sync after all content is in) 3. W1 starts after D1-D4 content is draft (can overlap with D5) 4. I1-I5 in any order (independent of each other) 5. W2 after D5 is complete (manual reflects final content) 6. W3 final verification

4. Concrete file changes

WP-DOC file changes

Path	Action	Scope
docs/ user_manual/ README.md	Edit (add sections in Parts III, IV, V, X)	Expand ~300-500 lines
docs/ user_manual/ SUMMARY.md	Edit (sync with README changes)	Update topic list
docs/ user_manual/ INDEX.md	Edit (sync with README changes)	Update keyword index
docs/ user_manual/ tutorials/ Tutorial_*.md	Edit (minor updates if needed)	Fix stale references
docs/ COMPLETE_*.md	Edit (minor updates if needed)	Fix stale config keys
docs/ user_manual/ examples/*.yaml	Edit (minor updates if needed)	Add new provider examples

WP-INST new files

Path	Action	Scope
scripts/ install.sh	Create	Platform-agnostic installer
scripts/ install.ps1	Create	Windows PowerShell installer
packaging/ debian/	Create	Debian packaging (control, postinst, prerm, service)
packaging/rpm/	Create	RPM packaging (spec, service)
packaging/ homebrew/ helixcode.rb	Create	Homebrew formula
packaging/ windows/	Create	NSIS/WiX installer config
packaging/ docker/ Dockerfile	Create	Minimal production Dockerfile
Makefile	Edit	

Path	Action	Scope
		Add make installers target

WP-WEB file changes

Path	Action	Scope
github_pages_website/docs/index.html	Edit	Update stats, links, feature list
github_pages_website/docs/manual/index.html	Edit	Sync with user manual
github_pages_website/docs/courses/	Edit	Update if curriculum stale
github_pages_website/docs/mobile/	Edit	Update if mobile info stale

5. Evidence & verification

Each work package MUST produce its own evidence:

- **WP-DOC:** grep for each expansion plan topic in README.md to confirm it exists. No “TODO” or “Coming soon” markers.
- **WP-INST:** Each package installs on a clean OS of the target type. make installers builds all. Verify with docker run test.
- **WP-WEB:** Test scripts pass. Manual walk-through of top 10 pages confirms no 404s. Stats match grep -c counts from actual code.

Phase 5 is DONE when: - ☐ USER_MANUAL_EXPANSION_PLAN.md all checkboxes ticked - ☐ make installers produces .deb, .rpm, .rb, .exe/.msi - ☐ curl -fsSL https://helixcode.dev/install.sh | sh works on Linux/macOS - ☐ Website test suite passes with 0 failures - ☐ Website feature list matches actual codebase features